Rockchip PLL Frequency Spreading Function Detailed Explanation

ID: RK-KF-YF-174

Release Version: V1.10.0

Release Date: 2026-01-09

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Product Version

Chipset	Kernel Version
RK3399	Linux 4.4 and above
RK1808	Linux 4.4 and above
RK3328	Linux 4.4 and above
RK3308	Linux 4.4 and above
RV1126	Linux 4.4 and above
PX30	Linux 4.4 and above
RK356X	Linux 4.19 and above
RK3588	Linux 5.10
RK3528	Linux 4.19
RK3562	Linux 5.10
RK3576	Linux 6.1
RK2118	RTT
RV1103B	Linux 5.10
RK3506	Linux 6.1
RV1126B/RV1126B-P	Linux 6.1
RK2116	RTT
RK1820	RTT
RK3538/9	linux6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Zhang Qing	2019-03-25	First version release
V1.1.0	Zhang Qing	2020-07-25	Added support for more chips
V1.2.0	Zhang Qing	2020-12-22	Added support for more chips
V1.2.1	Huang Ying	2021-04-26	Added document identifier, modified format
V1.3.0	Zhang Qing	2021-12-29	Added RK3588 support
V1.4.0	Zhang Qing	2022-12-14	Added RK3528 support
V1.5.0	Zhang Qing	2023-02-01	Added RK3562 support
V1.6.0	Zhang Qing	2024-03-21	Added RK3576, RK2118 support
V1.7.0	Zhang Qing	2024-07-21	Added RK3506, RV1103B support
V1.8.0	Zhang Qing	2025-04-16	Added RV1126B/RV1126B-P support
V1.9.0	Zhang Qing	2025-07-18	Added RK2116/RK1820 support
V1.10.0	Zhang Qing	2026-01-09	Added RK3538/9 support

Contents

Rockchip PLL Frequency Spreading Function Detailed Explanation

1. Spread Spectrum Concept Introduction
    1.1 Concept Explanation
    1.2 Working Principle
    1.3 Important Parameters
2. Spread Spectrum Parameter Configuration
    2.1 Introduction to PLL Spread Spectrum Parameters
    2.2 PLL Frequency Spreading Examples
    2.3 PLL Spread Spectrum Code Modification
3. Spread Spectrum Considerations

1. Spread Spectrum Concept Introduction

1.1 Concept Explanation

With the advancement of technology, the clock frequency of digital signals is increasing, and the circuit system poses higher and higher demands on signal setup, hold times, clock jitter, and other elements. EMI, or electromagnetic interference, refers to the impact of a circuit system on surrounding circuit systems through conduction or radiation. EMI can cause a reduction in circuit performance, and in severe cases, may lead to the failure of the entire system. In actual use, relevant authorities issue electromagnetic compatibility standards to ensure that electronic products on the market meet the regulatory requirements. Clock signals are often the highest frequency and steepest edge signals in the circuit system, and most EMI problems are related to clock signals.

There are many methods to reduce EMI, including shielding, filtering, isolation, ferrite magnetic rings, signal edge control, and adding power and GND layers in the PCB, among others. These methods can be flexibly applied in practice. Shielding is a relatively simple mechanical method with high costs and is not suitable for handheld and portable devices; filtering and signal edge control are effective for low-frequency signals and are not suitable for the widely used high-speed signals of today. Additionally, using EMI/RFI filters, which are passive components, increases costs; reducing EMI through LAYOUT techniques is obviously time-consuming, and the methods vary depending on the design.

We use frequency modulation to disperse the energy concentrated in a narrow frequency band to a set wide frequency band (Spread Spectrum, hereinafter referred to as SSG), thereby reducing the peak value of system electromagnetic radiation by lowering the amplitude (energy) of the clock at the fundamental frequency and odd harmonic frequencies. Generally, digital clocks have a high Q value, meaning all energy is concentrated in a very narrow frequency range, manifesting as a relatively high energy peak value. On the spectrum, it is easy to see a high peak value at the middle frequency and a lower peak value at the odd harmonic positions; SSCG reduces the peak energy by increasing the clock bandwidth, thereby reducing the Q value of the clock.

1.2 Working Principle

Spread spectrum clocking modulates the original clock signal through specific methods.
Application Features:
SSG is an active and cost-effective solution to EMI issues, which can address EMI problems across a broader frequency range while ensuring the integrity of the clock signal. Compared to traditional methods of suppressing EMI using Ferrite Beads and RF Chokes, SSC achieves the purpose of suppressing EMI peaks by modulating the frequency within the clock's integrated circuit. SSG not only modulates the clock source but also other data, address, and control signals synchronized with the clock source. These signals are modulated simultaneously with the clock spreading, resulting in a reduction of the overall EMI peaks. Therefore, clock spreading is a system-level solution. This is the greatest advantage of SSCG compared to other EMI suppression measures. The SSCG function can be configured by users to select different settings and modulation ranges. Without spreading, energy is concentrated; with spreading enabled, the energy in the middle is dispersed, reducing the peak values.

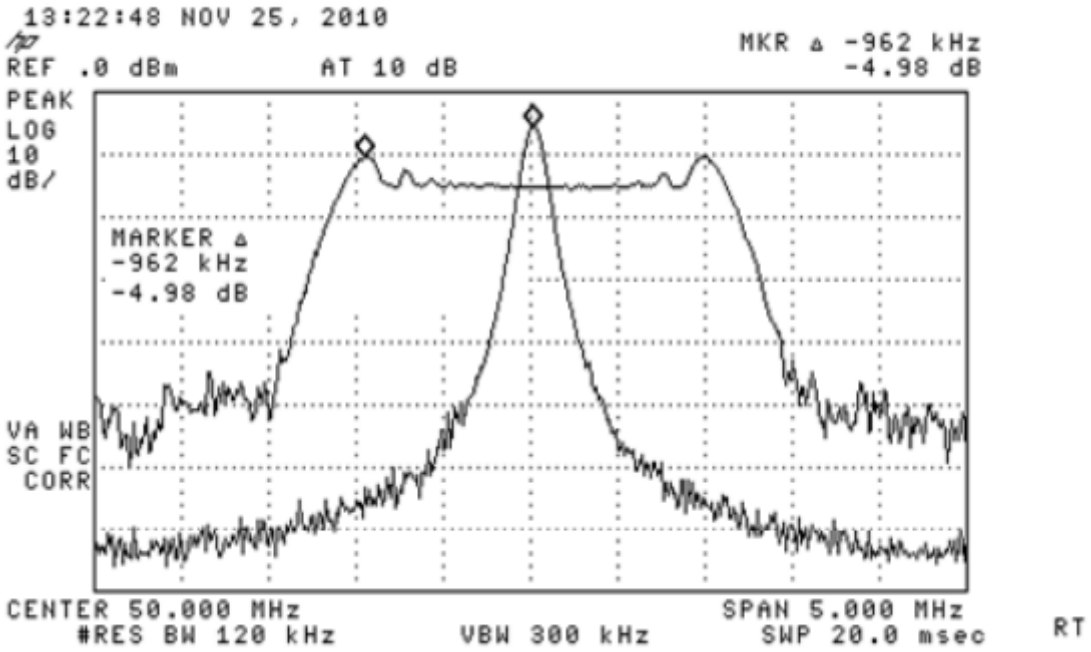


Figure 1 illustrates the working principle of SSG.

1.3 Important Parameters

Spread spectrum clocking has three main control parameters: Modulation Rate and Modulation Depth.

1. Modulation Rate

Modulation Rate refers to the rate at which the output clock frequency (f_o) varies within the set modulation frequency range. The Modulation Rate should be significantly lower than the source clock frequency (f_c) to avoid timing issues (setup/hold times, etc.), and it should be above the frequency range of human-recognizable sound (20Hz~20KHz) to prevent noise generation. In practical applications, the Modulation Rate is typically chosen between 30KHz and 120KHz.

2. Modulation Depth

Modulation Depth refers to the magnitude by which the spread spectrum output clock frequency (f_o) deviates from the source clock frequency (f_c) at the Modulation Rate (MR). Modulation Depth is expressed as a percentage (\%) of the deviation ((\Delta f)) from the source clock frequency. Modulation Depth determines the extent to which EMI peak values are reduced. Generally, the greater the Modulation Depth, the lower the EMI

peak values. When applying this, it is necessary to reasonably anticipate the frequency modulation range that the system can accept.

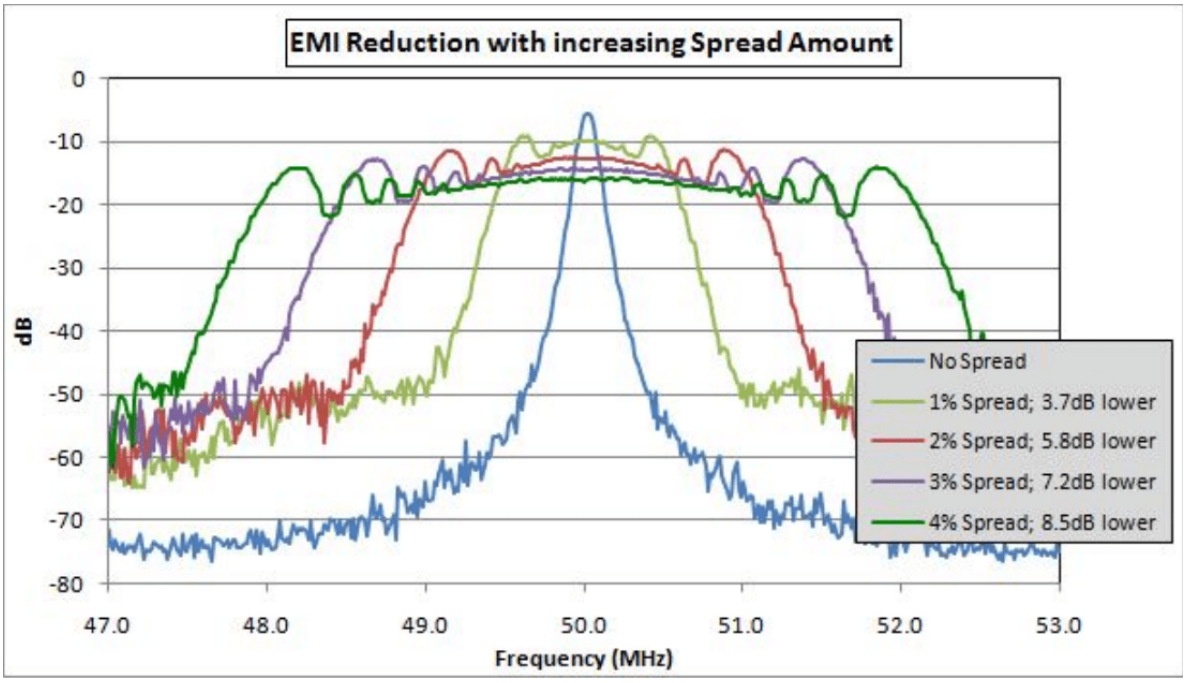


Figure 2 shows a comparison of EMI peak values at different Modulation Depths.

2. Spread Spectrum Parameter Configuration

2.1 Introduction to PLL Spread Spectrum Parameters

I. General PLL (except RK3588)

1. Enable Spread Spectrum Function of PLL

   (1) Configure dsmpd to 0, which means PLL needs to operate in fractional mode. If PLL does not have a fractional mode, it does not support spread spectrum functionality.

   (2) Configure ssmod_reset to 0, which means no reset modulator.

   (3) Configure ssmod_disable_sscg to 0, which means no bypass SSMOD by module.

   (4) Configure ssmod_bp to 0, which means no Bypass SSMOD by integration.

2. Configure PLL Spread Spectrum Amplitude

   Configure ssmod_spread to n, where n can take values from 0 to 1f, with n representing the spread spectrum amplitude of 0.n%.

3. Configure PLL Spread Spectrum Rate

   Configure ssmod_divval to n, where n can take values from 0 to f.

   Calculation formula for the rate:
   Spread spectrum rate (Hz) = 24000000/refdiv/128/n;

   refdiv is one of the PLL parameters, and the register can be consulted for detailed information in the datasheet. Generally, this value is 1.

4. Configure PLL Vector Table

Only RK3399 requires configuration of this parameter; other platforms can use the default internal vector table.

RK3399 configuration of vector table:
Configure ssmod_sel_ext_wave to 1, selecting the external table. Then configure ssmod_ext_maxaddr to ff.

Remark:
All parameters mentioned above can be directly queried in the TRM.

II. Special PLL (RK3588)

1. Enable Spread Spectrum Function of PLL

   Configure sscg_en to enable spread spectrum.

   0: disable dithered mode; 1: enable dithered mode.

2. Configure PLL Spread Spectrum Amplitude

   Configure mrr to n, where n can take values from 0 to 63, with n representing the spread spectrum amplitude of 0.n%.

   MR = mfr*mrr/m/32;

3. Configure PLL Spread Spectrum Rate

   Configure mfr to n, where n can take values from 0 to 255.

   MF = Fin/p/mfr/32;

   m and p are parameters of PLL, and the registers can be consulted for detailed information in the datasheet.

4. Configure Spread Spectrum Mode

   Configure sel_pf to set the spread spectrum mode.

   0: down spread; 1: up spread; 1x: center spread

   Considering jitter, it is generally recommended to choose center spread.

   Remark:
   All parameters mentioned above can be directly queried in the TRM.

2.2 PLL Frequency Spreading Examples

1. RK3399, GPLL Frequency Spreading

```
io -4 0xff760094 0x00010001 # Select external table to control MAX_ADDR
io -4 0xff760090 0x1f000100 # Adjust the gear [0, 1f]
io -4 0xff760090 0x00f000f0 # Adjust the period [0, f]
io -4 0xff760094 0xff00ff00 # Adjust MAX_ADDR [0, ff]
io -4 0xff76008c 0x00080000 # Enable fractional mode
io -4 0xff760090 0x00070000 # Enable frequency spreading function
```

2. RK3328, CPLL Frequency Spreading

```
io -4 0xff44004c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff44004c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff44004c 0x00080000
io -4 0xff440050 0x00070000
```

3. PX30, GPLL Frequency Spreading

```
io -4 0xff2bc00c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff2bc00c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff2bc004 0x10000000 # Enable fractional mode
io -4 0xff2bc00c 0x00070000 # Enable frequency spreading function
```

4. RK3308, VPLL0 Frequency Spreading

```
io -4 0xff50004c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff50004c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff500044 0x10000000 # Enable fractional mode
io -4 0xff50004c 0x00070000 # Enable frequency spreading function
```

5. RV1126, HPLL Frequency Spreading

```
io -4 0xff49006c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff49006c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff490064 0x10000000 # Enable fractional mode
io -4 0xff49006c 0x00070000 # Enable frequency spreading function
```

6. RK1808, GPLL Frequency Spreading

```
io -4 0xff35006c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff35006c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff350064 0x10000000 # Enable fractional mode
io -4 0xff35006c 0x00070000 # Enable frequency spreading function
```

7. RK356X, GPLL Frequency Spreading

```
io -4 0xfdd2004c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xfdd2004c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xfdd20044 0x10000000 # Enable fractional mode
io -4 0xfdd2004c 0x00070000 # Enable frequency spreading function
```

8. RK3588, GPLL Frequency Spreading

```
io -4 0xfd7c01cc 0x00ff000c # Set rate to 30khz
io -4 0xfd7c01cc 0x3f000500 # Set amplitude to 0.5%
io -4 0xfd7c01cc 0xc0008000 # Set frequency spreading mode
io -4 0xfd7c01d0 0x00010001 # Enable frequency spreading function
```

9. RK3528, GPLL Frequency Spreading

```
io -4 0xff4a006c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff4a006c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff4a006c 0x00080008 # Select central frequency spreading
io -4 0xff4a0064 0x10000000 # Enable fractional mode
io -4 0xff4a006c 0x00070000 # Enable frequency spreading function
```

10. RK3562, GPLL Frequency Spreading

```
io -4 0xff10006c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff10006c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff10006c 0x00080008 # Select central frequency spreading
io -4 0xff100064 0x10000000 # Enable fractional mode
io -4 0xff10006c 0x00070000 # Enable frequency spreading function
```

11. RK3576, GPLL Frequency Spreading

```
io -4 0x272001cc 0x00ff000c # Set rate to 30khz
io -4 0x272001cc 0x3f000500 # Set amplitude to 0.5%
io -4 0x272001cc 0xc0008000 # Set frequency spreading mode
io -4 0x272001d0 0x00010001 # Enable frequency spreading function
```

12. RK2118, V0PLL Frequency Spreading

```
io -4 0x50b1002c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0x50b1002c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0x50b1002c 0x00080008 # Select central frequency spreading
io -4 0x50b10024 0x10000000 # Enable fractional mode
io -4 0x50b1002c 0x00070000 # Enable frequency spreading function
```

13. RV1103B, GPLL Frequency Spreading

```
io -4 0x2006006c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0x2006006c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0x2006006c 0x00080008 # Select central frequency spreading
io -4 0x20060064 0x10000000 # Enable fractional mode
io -4 0x2006006c 0x00070000 # Enable frequency spreading function
```

14. RK3506, GPLL Frequency Spreading

```
io -4 0xff9b000c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xff9b000c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xff9b000c 0x00080008 # Select central frequency spreading
io -4 0xff9b0004 0x10000000 # Enable fractional mode
io -4 0xff9b000c 0x00070000 # Enable frequency spreading function
```

15. RV1126B/RV1126B-P, GPLL Frequency Spreading

```
io -4 0x2000002c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0x2000002c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0x2000002c 0x00080000 # Select central frequency spreading
io -4 0x20000024 0x10000000 # Enable fractional mode
io -4 0x2000002c 0x00070000 # Enable frequency spreading function
```

16. RK2116, V0PLL Frequency Spreading

```
io -4 0x4088002c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0x4088002c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0x4088002c 0x00080000 # Select central frequency spreading
io -4 0x40880024 0x10000000 # Enable fractional mode
io -4 0x4088002c 0x00070000 # Enable frequency spreading function
```

17. RK1820, GPLL Frequency Spreading

```
io -4 0x103080006c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0x103080006c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0x103080006c 0x00080000 # Select central frequency spreading
io -4 0x1030800064 0x10000000 # Enable fractional mode
io -4 0x103080006c 0x00070000 # Enable frequency spreading function
```

18. RK3538/9, GPLL Frequency Spreading

```
io -4 0xfd07004c 0x1f000800 # Set amplitude, where 8 can be 0-1f, n is 0.n%
io -4 0xfd07004c 0x00f00060 # Set rate to 30khz, where 6 can be 0-f
io -4 0xfd07004c 0x00080000 # Select central frequency spreading
io -4 0xfd070044 0x10000000 # Enable fractional mode
io -4 0xfd07004c 0x00070000 # Enable frequency spreading function
```

Note:
RK3036\RK312X\RK322X\RK3188\RK3288\RK3368 do not support frequency spreading function.
For other PLLs with similar processing, only the base address needs to be changed to the corresponding PLL.
It is recommended to use a frequency spreading rate of 30Khz, but there is no reference value for the amplitude of frequency spreading, which needs to be measured in the laboratory to determine whether EMI certification is passed.

2.3 PLL Spread Spectrum Code Modification

If the spread spectrum parameters are determined, software modification is required. It is recommended to configure the spread spectrum parameters as early as possible. Therefore, two examples are provided, one for handling in UBOOT and the other for handling in the kernel.

1. UBOOT (RK3399 DPLL Spread Spectrum)

```
diff --git a/drivers/clk/rockchip/clk_rk3399.c
b/drivers/clk/rockchip/clk_rk3399.c
--- a/drivers/clk/rockchip/clk_rk3399.c
+++ b/drivers/clk/rockchip/clk_rk3399.c
@@ -1479,6 +1479,15 @@ static int rk3399_clk_probe(struct udevice *dev)
 {
    struct rk3399_clk_priv *priv = dev_get_priv(dev);
+   /* Spread spectrum amplitude */
+   writel(0x1f001f00, &priv->cru->dpll_con[4]);
+   /* Spread spectrum period */
+   writel(0x00f00060, &priv->cru->dpll_con[4]);
+   writel(0x7f002700, &priv->cru->dpll_con[5]);
+   writel(0x00080000, &priv->cru->dpll_con[3]);
+   writel(0x00070000, &priv->cru->dpll_con[4]);
+   writel(0x00010001, &priv->cru->dpll_con[5]);
+
 #if CONFIG_IS_ENABLED(OF_PLATDATA)
    struct rk3399_clk_plat *plat = dev_get_platdata(dev);
```

2. KERNEL (RK3399 GPLL VPLL Spread Spectrum)

```
diff --git a/drivers/clk/rockchip/clk-rk3399.c b/drivers/clk/rockchip/clk-
rk3399.c
--- a/drivers/clk/rockchip/clk-rk3399.c
+++ b/drivers/clk/rockchip/clk-rk3399.c
@@ -1766,6 +1766,20 @@ static void __init rk3399_clk_init(struct device_node
*np)

     rockchip_register_restart_notifier(ctx, RK3399_GLB_SRST_FST, NULL);
+    writel(0x00010001, 0xff760094);
+    writel(0x1f000100, 0xff760090);
+    writel(0x00f000f0, 0xff760090);
+    writel(0x7f002700, 0xff760094);
+    writel(0x00070000, 0xff760090);
+    writel(0x00080000, 0xff76008c);
+
+    writel(0x00010001, 0xff7600d4);
+    writel(0x1f000a00, 0xff7600d0);
+    writel(0x00f00060, 0xff7600d0);
+    writel(0x7f002700, 0xff7600d4);
+    writel(0x00070000, 0xff7600d0);
+    writel(0x00080000, 0xff7600cc);
+
     rockchip_clk_of_add_provider(np, ctx);
 }
 CLK_OF_DECLARE(rk3399_cru, "rockchip,rk3399-cru", rk3399_clk_init);
```

Note:
If it is DDR (DPLL) spread spectrum, it is best to handle it in the LOADER because handling DDR clock in other locations may cause system instability, crashes, and other issues.

3. Spread Spectrum Considerations

1. Clock Jitter

One significant drawback of clock spreading is its inapplicability to applications sensitive to clock precision, such as Ethernet and CAN bus. When selecting clock spreading and modulation depth, designers must pay special attention to the additional Jitter introduced by spreading, which may lead to establishment/hold time issues, high error rates, and PLL lock loss problems. Jitter is categorized differently and affects system performance to varying degrees.
Clock spreading introduces very little period jitter. For instance, a clock spread with a modulation speed of 30KHz to 120KHz requires the source clock frequency to be at least 1000 times the MR, meaning that it takes at least 1000 clock cycles to complete one modulation cycle, resulting in a very small difference between adjacent cycles of the output clock fc. Of course, the device (clock generator) itself also has inherent period jitter. Spreading introduces approximately less than 0.05% period jitter into the system. Therefore, spreading is very suitable for systems that require low period jitter, low error rates, and low EMI.

2. Stability

Since our platform utilizes PLL spreading, once PLL spreading is enabled, all self-clocks below the PLL operate in spread mode. The impact on self-clocks is largely unknown, necessitating increased functional and stability testing.

3. Commonality

Since our platform utilizes PLL spreading, once PLL spreading is enabled, all self-clocks below the PLL operate in spread mode. For example, EMMC and SDIO are both under one PLL, but they have different EMI requirements for spreading parameters. When sharing one PLL, special attention must be paid to choose a compromise spreading parameter that can meet the EMI requirements of both modules.

Notes:

- DDR, USB, SDIO, I2C, SPI, I2S, USB3.0, PCIE, eDP, etc., generally have defined SSC in their specifications; parameters should be set as required. For those not defined in the specifications and where enabling SSC amplitude leads to partial test failures, stability and compatibility testing are required.
- Only enable SSC as needed, with the principle of keeping the SSC amplitude as small as possible while meeting EMI requirements.

For example: USB2.0, after enabling SSC, most frequency test items will not pass. In this case, it is necessary to look at the size of the flexible buffer in the PHY; the larger the buffer, the stronger the adaptability to the spread amplitude, and the better the compatibility. Conversely, the smaller the buffer, the weaker the adaptability to the spread amplitude, which can easily lead to buffer overflow during high-frequency spreading stages.

- Therefore, spreading carries risks, and the phenomena after spreading will vary with each project, requiring case-by-case analysis.